ncuscc 考核感悟报告

一：关于环境

这个容器排队从 15 号开始我就拿不到了，一直显示有阴兵排队，这谁能绷住。所以除了读数统计（17）是排到了容器，根据模板写的，后面做的所有题目都是仅根据 hints 和 README 做了，可能和原题有所差别



正在焦急排队中

二：具体感悟

1. 读数统计（17）

上来直接写了 `int min = 0; int max = 0;`（顺手的事），大傻鱼还没看出来，结果一看提示不能用 0 初始化

初始化直接写 0 导致 0 参与进了统计，被当成已经读到的一个读数。只要 0 落在所有读数的范围之外（全正或全负），它就永远不会被替换掉：全负数时 max 一直是 0，全正数时 min 一直是 0。数据里有正有负时反而不容易发现，因为 0 正好在中间，会被真实的数顶掉。

2. 设备巡检记录（18）

n 行巡检日志按设备类型汇总成 4 行统计，再找出总工时最多的那一类。hints.md 这个第三个提示是 c++版本的，c 版本需要改写。std::string、成员默认初始化，C 里要改成 char[] 加 strcmp。

3. 固定容量读数窗口（19）

注意因为我排不到容器，所以题目这个修复缺陷我只能当滚木了。提示说不要只增加数组容量，原因是就算把 window[8]改成 window[100000],程序确实不会崩，但是 oldest 会变成 window[0],sum 变成全部数之和，样例 2 立刻错

4. 多文件统计工具（20）

这题把程序拆成三个文件：一个读数据，一个算统计，一个打印，与前面都是一个 main.c 写到底的题目不同。构建的时候报 undefined reference，错因是漏了一个文件。单文件的程序没有清单可漏，只有一份源码，编译器全看得见；拆成多个文件之后，构建时要显式写出哪些文件参与，少写一个，编译能过，链接的时候才报找不到函数。

5. 四线程快速排序观察

线程我理解的是操作系统调度的基本单位，我学过一点 Go,接触过 GMP 模型，就是复用操作系统线程的执行时间，动态分配给轻量级的 goroutine 任务，从而避免频繁的线程创建，销毁和上下文切换。

放到这题来看，我理解的 C 和 Go 的差别就是 C 既没有 p 这一层，也没有 G（goroutine），调度只有内核那一层。线程本身就是执行流，任务只是线程里跑的一段函数，任务结束线程也结束，没法留下来接下一个活。在 Go 中从来不用想开几个，但在 C 中需要把数量按住。

本地环境和沙盒不同，本地是 32 核，单 CPU 沙盒只有 1 核，导致开了 4 个线程程序可能更慢，开线程会有开销，4 个线程只是在 1 个核上轮流用，切换本身也有开销。

但是这道题虽然我只开了 2 个工作线程，并不是因为省开销。因为截止条件设在区间长度 4，16 个数切两刀就到头了，根本用不满 4 个。